SOFTWARE ENGINEERING LAB — LAB 4

Work Report: Mastermind Vibe Coding

Name	Abhirama H Y
SRN	PES2UG24CS023
Section	A
Lab Activity	Lab 4 — Vibe Coding (Mastermind)

1. Objective

The objective of this lab was to improve the assigned Mastermind game using AI-assisted programming (vibe coding), correct defects in the existing implementation, add requested features, test the changes, and document the work.

2. Work Completed

- **Reviewed the existing project:** Examined the provided Mastermind code and identified issues in how exact and partial matches were counted, particularly when symbols were repeated.
- **Corrected the feedback algorithm:** Updated `logic.py` to count exact matches first and calculate partial matches from the remaining unmatched symbols using frequency counts. This prevents duplicate symbols from being counted more times than they occur in the secret code.
- **Improved game lifecycle handling:** Updated `game.py` to track whether the game was finished or won, manage the number of turns, handle wins and losses, reveal the secret code after a loss or quit, and prevent guesses after the game ends.
- **Added difficulty levels:** Implemented Easy, Medium, and Hard modes with different code lengths, permitted symbol ranges, and numbers of allowed guesses.
- **Improved input validation:** Added checks for empty guesses, incorrect lengths, and symbols outside the selected difficulty's permitted range. Invalid guesses do not consume a turn or enter the guess history.
- **Added guess history:** Implemented a formatted history table showing each accepted guess and its exact and partial match counts. The history can also be requested during gameplay.
- **Created automated tests:** Added `test_logic.py` for scoring behaviour and `test_game.py` for game lifecycle, difficulty modes, input validation, and history behaviour.
- **Updated documentation:** Updated `README.md` with game rules, setup and run instructions, difficulty configurations, feedback logic, test commands, implemented requirements, and limitations.
- **Recorded evidence and AI interaction:** The repository contains before/after gameplay recordings and screenshots, along with a PDF containing prompts and Antigravity responses.

3. Main Features Implemented

- Duplicate-aware exact and partial match calculation.
- Win, loss, and quit handling with clear messages.
- Three difficulty modes: Easy, Medium, and Hard.
- Validation of guess length and permitted symbols.
- Guess history display during and after the game.

- Automated unit tests using Python's standard-library unittest framework.
- No third-party packages required according to the project documentation.

4. Testing

The project includes automated test files for both the scoring logic and overall game behaviour. The README documents a test suite of 27 cases and provides commands to run the complete suite or either test module. The reported test result is based on the project documentation; this report does not claim an independent test execution.

5. Deliverables Prepared

- Updated Mastermind source files: `main.py`, `game.py`, and `logic.py`.
- Automated tests: `test_logic.py` and `test_game.py`.
- Updated project documentation: `README.md` and `requirements.txt`.
- Before-change and after-change gameplay evidence, including videos and screenshots.
- PDF record of the prompts and Antigravity responses used during development.

6. Conclusion

The Mastermind project was enhanced to improve scoring correctness, input handling, gameplay flow, difficulty selection, and usability. Automated tests and documentation were added to support verification and explain the implemented features. The work demonstrates the use of AI-assisted coding alongside testing and documentation.

Prepared by Abhirama H Y | SRN: PES2UG24CS023 | Section: A

https://github.com/A6h1ramahy/PES2UG24CS023_SE-Lab

https://github.com/A6h1ramahy/PES2UG24CS023_SE-Lab/tree/main/LAB4